ABCI 실습 가이드

개요

이 문서에서는 실제로 동작하는 ABCI 애플리케이션을 단계별로 구현해보겠습니다. CometBFT의 KVStore 예제를 기반으로 하여 **간단한 키-값 저장소 애플리케이션**을 만들어보고, Query 메서드 활용 및 상태 조회 방법을 학습합니다.

실습 목표

1. **기본 ABCI 애플리케이션 구조** 이해
2. **핵심 ABCI 메서드** 구현
3. **상태 관리 및 영속성** 구현
4. **Query 메서드를 통한 상태 조회** 구현
5. **트랜잭션 검증 로직** 구현

1. 프로젝트 설정

필요한 의존성

```
// go.mod

module  simple-kvstore


go  1.21
```

```
require (
github.com/cometbft/cometbft  v2.0.0
github.com/cometbft/cometbft-db  v0.9.1
)
```

기본 구조 설정

```
// main.go
package main

import (
"flag"
"fmt"
"log"
"os"
"os/signal"
"path/filepath"
"syscall"

"github.com/cometbft/cometbft/v2/abci/server"
"github.com/cometbft/cometbft/v2/abci/types"
cmtlog  "github.com/cometbft/cometbft/v2/libs/log"
```

```
)

func main() {
var socketAddr string
var dbDir string
flag.StringVar(&socketAddr, "socket-addr", "unix://example.sock", "Unix domain s
flag.StringVar(&dbDir, "db-dir", "/tmp/kvstore", "Database directory")
flag.Parse()

// 애플리케이션 생성
app := NewKVStoreApp(dbDir)
// ABCI 서버 시작
server, err := server.NewServer(socketAddr, "socket", app)
if err != nil {
log.Fatal(err)
}
server.SetLogger(cmtlog.NewTMLogger(cmtlog.NewSyncWriter(os.Stdout)))
if err := server.Start(); err != nil {
log.Fatal(err)
}
defer server.Stop()

// 종료 신호 대기
```

```go
c := make(chan os.Signal, 1)

signal.Notify(c, os.Interrupt, syscall.SIGTERM)

<-c

}
```

2. 기본 애플리케이션 구조

상태 구조 정의

```go
// kvstore.go

package main


import (

"context"

"encoding/binary"

"encoding/json"

"fmt"

"strconv"

"strings"


dbm  "github.com/cometbft/cometbft-db"

"github.com/cometbft/cometbft/v2/abci/types"

"github.com/cometbft/cometbft/v2/libs/log"
```

```go
)

// 상태 키 정의
var (
stateKey = []byte("stateKey")
dataPrefix = []byte("data:")
)

// 응답 코드 정의
const (
CodeTypeOK  uint32 = 0
CodeTypeInvalidFormat  uint32 = 1
CodeTypeInvalidKey  uint32 = 2
CodeTypeNotFound  uint32 = 3
CodeTypeInternalError  uint32 = 4
)

// 애플리케이션 상태
type State struct {
db dbm.DB
Size  int64  `json:"size"`  // 처리된 트랜잭션 수
Height  int64  `json:"height"`  // 현재 블록 높이
}
```

```go
// KVStore 애플리케이션
type KVStoreApp struct {
types.BaseApplication
state State
logger log.Logger
}


// 새로운 KVStore 애플리케이션 생성
func  NewKVStoreApp(dbDir  string) *KVStoreApp {
// 데이터베이스 생성
db, err := dbm.NewGoLevelDB("kvstore", dbDir)
if  err != nil {
panic(fmt.Errorf("failed to create database: %w", err))
}
// 상태 로드
state := loadState(db)
return &KVStoreApp{
state: state,
logger: log.NewNopLogger(),
}
}
```

```go
// 상태 로드
func loadState(db dbm.DB) State {
var state State
state.db = db
stateBytes, err := db.Get(stateKey)
if err != nil {
panic(err)
}
if len(stateBytes) == 0 {
return state // 빈 상태로 시작
}
err = json.Unmarshal(stateBytes, &state)
if err != nil {
panic(err)
}
return state
}

// 상태 저장
func (s State) save() {
stateBytes, err := json.Marshal(s)
if err != nil {
panic(err)
```

```
}
err = s.db.Set(stateKey, stateBytes)
if  err != nil {
panic(err)
}
}

// 애플리케이션 해시 계산
func (s State) Hash() []byte {
appHash := make([]byte, 8)
binary.PutVarint(appHash, s.Size)
return  appHash
}
```

3. 핵심 ABCI 메서드 구현

Info 메서드

```
func (app *KVStoreApp) Info(ctx context.Context, req *types.InfoRequest) (*types
return &types.InfoResponse{
Data: fmt.Sprintf("KVStore v1.0 - Size: %d", app.state.Size),
Version: "1.0.0",
AppVersion: 1,
```

```
LastBlockHeight: app.state.Height,
LastBlockAppHash: app.state.Hash(),
}, nil
}
```

InitChain 메서드

```
func (app *KVStoreApp) InitChain(ctx context.Context, req *types.InitChainReques
app.logger.Info("Initializing chain", "chain_id", req.ChainId)
// 초기 상태 설정
app.state.Height = req.InitialHeight
app.state.save()
return &types.InitChainResponse{
AppHash: app.state.Hash(),
}, nil
}
```

CheckTx 메서드

```
func (app *KVStoreApp) CheckTx(ctx context.Context, req *types.CheckTxRequest) (
// 트랜잭션 파싱
key, value, err := parseTx(req.Tx)
if  err != nil {
```

```go
return &types.CheckTxResponse{
Code: CodeTypeInvalidFormat,
Log: err.Error(),
}, nil
}
// 키 유효성 검사
if len(key) == 0 {
return &types.CheckTxResponse{
Code: CodeTypeInvalidKey,
Log: "Key cannot be empty",
}, nil
}
// 키 길이 제한 (예: 최대 100바이트)
if len(key) > 100 {
return &types.CheckTxResponse{
Code: CodeTypeInvalidKey,
Log: "Key too long (max 100 bytes)",
}, nil
}
// 값 길이 제한 (예: 최대 1KB)
if len(value) > 1024 {
return &types.CheckTxResponse{
Code: CodeTypeInvalidFormat,
```

```go
Log: "Value too long (max 1KB)",
}, nil
}
return &types.CheckTxResponse{
Code: CodeTypeOK,
GasWanted: 1, // 간단한 가스 모델
}, nil
}

// 트랜잭션 파싱 함수
func parseTx(tx []byte) (key, value string, err error) {
parts := strings.SplitN(string(tx), "=", 2)
if len(parts) != 2 {
return "", "", fmt.Errorf("invalid transaction format, expected 'key=value'")
}
return parts[0], parts[1], nil
}
```

PrepareProposal 메서드

```go
func (app *KVStoreApp) PrepareProposal(ctx context.Context, req *types.PreparePr
var selectedTxs [][]byte
```

```go
var totalBytes int64
// 트랜잭션 선택 (크기 제한 고려)
for _, tx := range req.Txs {
txSize := int64(len(tx))
if totalBytes+txSize > req.MaxTxBytes {
break
}
// 트랜잭션 재검증
checkResp, err := app.CheckTx(ctx, &types.CheckTxRequest{
Tx: tx,
Type: types.CHECK_TX_TYPE_CHECK,
})
if err != nil {
continue
}
if checkResp.Code == CodeTypeOK {
selectedTxs = append(selectedTxs, tx)
totalBytes += txSize
}
}
return &types.PrepareProposalResponse{
Txs: selectedTxs,
}, nil
```

```
}
```

ProcessProposal 메서드

```
func (app *KVStoreApp) ProcessProposal(ctx context.Context, req *types.ProcessPr

// 모든 트랜잭션 검증

for  _, tx := range  req.Txs {

checkResp, err := app.CheckTx(ctx, &types.CheckTxRequest{

Tx: tx,

Type: types.CHECK_TX_TYPE_CHECK,

})

if  err != nil || checkResp.Code != CodeTypeOK {

return &types.ProcessProposalResponse{

Status: types.PROCESS_PROPOSAL_STATUS_REJECT,

}, nil

}

}

return &types.ProcessProposalResponse{

Status: types.PROCESS_PROPOSAL_STATUS_ACCEPT,

}, nil

}
```

FinalizeBlock 메서드

```go
func (app *KVStoreApp) FinalizeBlock(ctx context.Context, req *types.FinalizeBlo

var  txResults []*types.ExecTxResult

// 각 트랜잭션 실행

for  _, tx := range  req.Txs {

result := app.executeTx(tx)

txResults = append(txResults, result)

// 성공한 트랜잭션만 상태 크기 증가

if  result.Code == CodeTypeOK {

app.state.Size++

}

}

// 블록 높이 업데이트

app.state.Height = req.Height

return &types.FinalizeBlockResponse{

TxResults: txResults,

AppHash: app.state.Hash(),

}, nil

}



// 트랜잭션 실행

func (app *KVStoreApp) executeTx(tx []byte) *types.ExecTxResult {

// 트랜잭션 파싱
```

```go
key, value, err := parseTx(tx)
if  err != nil {
return &types.ExecTxResult{
Code: CodeTypeInvalidFormat,
Log: err.Error(),
}
}
// 데이터 저장
dataKey := append(dataPrefix, []byte(key)...)
err = app.state.db.Set(dataKey, []byte(value))
if  err != nil {
return &types.ExecTxResult{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Failed to store data: %v", err),
}
}
// 성공 응답 및 이벤트 생성
return &types.ExecTxResult{
Code: CodeTypeOK,
Events: []types.Event{
{
Type: "kvstore.set",
Attributes: []types.EventAttribute{
```

```
{Key: "key", Value: key, Index: true},
{Key: "value", Value: value, Index: true},
{Key: "action", Value: "set", Index: true},
},
},
},
}
}
```

Commit 메서드

```
func (app *KVStoreApp) Commit(ctx context.Context, req *types.CommitRequest) (*t
// 상태를 디스크에 저장
app.state.save()
app.logger.Info("Committed state",
"height", app.state.Height,
"size", app.state.Size,
"hash", fmt.Sprintf("%X", app.state.Hash()))
return &types.CommitResponse{}, nil
}
```

4. Query 메서드 구현 및 활용

기본 Query 메서드

```
func (app *KVStoreApp) Query(ctx context.Context, req *types.QueryRequest) (*typ

switch  req.Path {

case  "/store":

return  app.queryStore(req)

case  "/info":

return  app.queryInfo(req)

case  "/stats":

return  app.queryStats(req)

default:

return &types.QueryResponse{

Code: CodeTypeNotFound,

Log: fmt.Sprintf("Unknown query path: %s", req.Path),

}, nil

}

}


// 키-값 저장소 쿼리

func (app *KVStoreApp) queryStore(req *types.QueryRequest) (*types.QueryResponse

key := req.Data

if  len(key) == 0 {

return &types.QueryResponse{
```

```
Code: CodeTypeInvalidKey,

Log: "Key cannot be empty",

}, nil

}

// 데이터 조회

dataKey := append(dataPrefix, key...)

value, err := app.state.db.Get(dataKey)

if  err != nil {

return &types.QueryResponse{

Code: CodeTypeInternalError,

Log: fmt.Sprintf("Database error: %v", err),

}, nil

}

if  value == nil {

return &types.QueryResponse{

Code: CodeTypeNotFound,

Log: "Key not found",

Key: key,

Height: app.state.Height,

}, nil

}

return &types.QueryResponse{

Code: CodeTypeOK,
```

```go
Log: "Key found",
Key: key,
Value: value,
Height: app.state.Height,
}, nil
}


// 애플리케이션 정보 쿼리
func (app *KVStoreApp) queryInfo(req *types.QueryRequest) (*types.QueryResponse,
info := map[string]interface{}{
"height": app.state.Height,
"size": app.state.Size,
"hash": fmt.Sprintf("%X", app.state.Hash()),
}
infoBytes, err := json.Marshal(info)
if  err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Failed to marshal info: %v", err),
}, nil
}
return &types.QueryResponse{
Code: CodeTypeOK,
```

```go
Value: infoBytes,
Height: app.state.Height,
}, nil
}


// 통계 정보 쿼리
func (app *KVStoreApp) queryStats(req *types.QueryRequest) (*types.QueryResponse
// 저장된 키의 개수 계산
keyCount := int64(0)
iterator, err := app.state.db.Iterator(dataPrefix, nil)
if err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Iterator error: %v", err),
}, nil
}
defer iterator.Close()
for ; iterator.Valid(); iterator.Next() {
keyCount++
}
stats := map[string]interface{}{
"total_keys": keyCount,
"total_transactions": app.state.Size,
```

```
"current_height": app.state.Height,
}
statsBytes, err := json.Marshal(stats)
if  err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Failed to marshal stats: %v", err),
}, nil
}
return &types.QueryResponse{
Code: CodeTypeOK,
Value: statsBytes,
Height: app.state.Height,
}, nil
}
```

5. 고급 기능 구현

범위 쿼리 지원

```
// 범위 쿼리 (/range?start=key1&end=key2)
func (app *KVStoreApp) queryRange(req *types.QueryRequest) (*types.QueryResponse
// 쿼리 파라미터 파싱
```

```go
params := parseQueryParams(string(req.Data))
startKey := params["start"]
endKey := params["end"]
limitStr := params["limit"]
limit := 100  // 기본 제한
if  limitStr != "" {
if  l, err := strconv.Atoi(limitStr);  err == nil && l > 0 {
limit = l
}
}
var  results []map[string]string
count := 0
// 범위 이터레이터 생성
startPrefix := append(dataPrefix, []byte(startKey)...)
endPrefix := append(dataPrefix, []byte(endKey)...)
iterator, err := app.state.db.Iterator(startPrefix, endPrefix)
if  err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Iterator error: %v", err),
}, nil
}
defer  iterator.Close()
```

```
for ; iterator.Valid() && count < limit; iterator.Next() {
key := string(iterator.Key()[len(dataPrefix):]) // 프리픽스 제거
value := string(iterator.Value())
results = append(results, map[string]string{
"key": key,
"value": value,
})
count++
}
response := map[string]interface{}{
"results": results,
"count": count,
"limit": limit,
}
responseBytes, err := json.Marshal(response)
if err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Failed to marshal response: %v", err),
}, nil
}
return &types.QueryResponse{
Code: CodeTypeOK,
```

```go
Value: responseBytes,
Height: app.state.Height,
}, nil
}

// 쿼리 파라미터 파싱
func parseQueryParams(query string) map[string]string {
params := make(map[string]string)
pairs := strings.Split(query, "&")
for _, pair := range pairs {
kv := strings.SplitN(pair, "=", 2)
if len(kv) == 2 {
params[kv[0]] = kv[1]
}
}
return params
}
```

트랜잭션 히스토리 추적

```go
// 트랜잭션 히스토리 저장
type TxHistory struct {
Height int64 `json:"height"`
```

```
TxIndex int `json:"tx_index"`
Key string `json:"key"`
Value string `json:"value"`
Timestamp int64 `json:"timestamp"`
}

// 히스토리 저장 (FinalizeBlock에서 호출)
func (app *KVStoreApp) saveHistory(height int64, txIndex int, key, value stri
history := TxHistory{
Height: height,
TxIndex: txIndex,
Key: key,
Value: value,
Timestamp: time.Now().Unix(),
}
historyBytes, err := json.Marshal(history)
if err != nil {
app.logger.Error("Failed to marshal history", "error", err)
return
}
// 히스토리 키: history:{height}:{txIndex}
historyKey := fmt.Sprintf("history:%d:%d", height, txIndex)
err = app.state.db.Set([]byte(historyKey), historyBytes)
```

```go
if err != nil {
app.logger.Error("Failed to save history", "error", err)
}
}


// 히스토리 쿼리
func (app *KVStoreApp) queryHistory(req *types.QueryRequest) (*types.QueryRespon
key := string(req.Data)
var histories []TxHistory
// 모든 히스토리 검색
iterator, err := app.state.db.Iterator([]byte("history:"), nil)
if err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Iterator error: %v", err),
}, nil
}
defer iterator.Close()
for ; iterator.Valid(); iterator.Next() {
var history TxHistory
err := json.Unmarshal(iterator.Value(), &history)
if err != nil {
continue
```

```
}
if  history.Key == key {
histories = append(histories, history)
}
}
responseBytes, err := json.Marshal(histories)
if  err != nil {
return &types.QueryResponse{
Code: CodeTypeInternalError,
Log: fmt.Sprintf("Failed to marshal histories: %v", err),
}, nil
}
return &types.QueryResponse{
Code: CodeTypeOK,
Value: responseBytes,
Height: app.state.Height,
}, nil
}
```

6. 클라이언트 사용 예제

Go 클라이언트

```go
// client.go
package main

import (
"context"
"fmt"
"log"

"github.com/cometbft/cometbft/v2/rpc/client/http"
)

func main() {
// CometBFT RPC 클라이언트 생성
client, err := http.New("http://localhost:26657", "/websocket")
if err != nil {
log.Fatal(err)
}
ctx := context.Background()
// 1. 데이터 저장
tx := []byte("mykey=myvalue")
result, err := client.BroadcastTxCommit(ctx, tx)
if err != nil {
```

```go
log.Fatal(err)
}
fmt.Printf("Transaction result: %+v\n", result)
// 2. 데이터 조회
queryResult, err := client.ABCIQuery(ctx, "/store", []byte("mykey"))
if err != nil {
log.Fatal(err)
}
if queryResult.Response.Code == 0 {
fmt.Printf("Value: %s\n", string(queryResult.Response.Value))
} else {
fmt.Printf("Query failed: %s\n", queryResult.Response.Log)
}
// 3. 통계 조회
statsResult, err := client.ABCIQuery(ctx, "/stats", nil)
if err != nil {
log.Fatal(err)
}
if statsResult.Response.Code == 0 {
fmt.Printf("Stats: %s\n", string(statsResult.Response.Value))
}
}
```

커맨드라인 도구

```
#!/bin/bash

# kvstore-cli.sh


COMETBFT_RPC="http://localhost:26657"


case "$1" in

"set")

if [ -z "$2" ] || [ -z "$3" ]; then

echo "Usage: $0 set <key> <value>"

exit 1

fi

curl -s "$COMETBFT_RPC/broadcast_tx_commit?tx=\"$2=$3\"" | jq .

;;

"get")

if [ -z "$2" ]; then

echo "Usage: $0 get <key>"

exit 1

fi

curl -s "$COMETBFT_RPC/abci_query?path=\"/store\"&data=\"$2\"" | jq -r '.res

;;

"stats")
```

```
curl -s "$COMETBFT_RPC/abci_query?path=\"/stats\"" | jq -r '.result.response
;;
"info")
curl -s "$COMETBFT_RPC/abci_query?path=\"/info\"" | jq -r '.result.response.
;;
*)
echo "Usage: $0 {set|get|stats|info} [args...]"
exit 1
;;
esac
```

7. 테스트 작성

단위 테스트

```
// kvstore_test.go
package main

import (
"context"
"testing"

"github.com/cometbft/cometbft/v2/abci/types"
```

```go
"github.com/stretchr/testify/assert"
"github.com/stretchr/testify/require"
)

func TestKVStoreApp(t *testing.T) {
app := NewKVStoreApp("/tmp/kvstore-test")
defer app.state.db.Close()
ctx := context.Background()
// InitChain 테스트
_, err := app.InitChain(ctx, &types.InitChainRequest{
ChainId: "test-chain",
InitialHeight: 1,
})
require.NoError(t, err)
// CheckTx 테스트
checkResp, err := app.CheckTx(ctx, &types.CheckTxRequest{
Tx: []byte("testkey=testvalue"),
})
require.NoError(t, err)
assert.Equal(t, uint32(0), checkResp.Code)
// FinalizeBlock 테스트
finalizeResp, err := app.FinalizeBlock(ctx, &types.FinalizeBlockRequest{
Height: 1,
```

```go
Txs: [][]byte{[]byte("testkey=testvalue")},
})
require.NoError(t, err)
assert.Equal(t, 1, len(finalizeResp.TxResults))
assert.Equal(t, uint32(0), finalizeResp.TxResults[0].Code)
// Commit 테스트
_, err = app.Commit(ctx, &types.CommitRequest{})
require.NoError(t, err)
// Query 테스트
queryResp, err := app.Query(ctx, &types.QueryRequest{
Path: "/store",
Data: []byte("testkey"),
})
require.NoError(t, err)
assert.Equal(t, uint32(0), queryResp.Code)
assert.Equal(t, "testvalue", string(queryResp.Value))
}


func TestInvalidTransactions(t *testing.T) {
app := NewKVStoreApp("/tmp/kvstore-test-invalid")
defer app.state.db.Close()
ctx := context.Background()
testCases := []struct {
```

```
name  string

tx []byte

code  uint32

}{

{"empty key", []byte("=value"), CodeTypeInvalidKey},

{"no equals", []byte("keyvalue"), CodeTypeInvalidFormat},

{"multiple equals", []byte("key=val=ue"), CodeTypeInvalidFormat},

{"too long key", []byte(strings.Repeat("a", 101) + "=value"), CodeTypeInvalidKey

}

for  _, tc := range  testCases {

t.Run(tc.name, func(t *testing.T) {

resp, err := app.CheckTx(ctx, &types.CheckTxRequest{Tx: tc.tx})

require.NoError(t, err)

assert.Equal(t, tc.code, resp.Code)

})

}

}
```

8. 실행 및 테스트

애플리케이션 실행

```
# 1. 애플리케이션 빌드
```

```
go build -o kvstore-app


# 2. CometBFT 초기화
cometbft init


# 3. 애플리케이션 시작
./kvstore-app --socket-addr unix://kvstore.sock --db-dir ./data


# 4. 다른 터미널에서 CometBFT 시작
cometbft node --proxy_app unix://kvstore.sock
```

기능 테스트

```
# 데이터 저장
curl "http://localhost:26657/broadcast_tx_commit?tx=\"hello=world\""


# 데이터 조회
curl "http://localhost:26657/abci_query?path=\"/store\"&data=\"hello\""


# 통계 조회
curl "http://localhost:26657/abci_query?path=\"/stats\""


# 애플리케이션 정보 조회
```

```
curl "http://localhost:26657/abci_query?path=\"/info\""
```

9. 성능 최적화 팁

1. 데이터베이스 최적화

```
// 배치 쓰기 사용
func (app *KVStoreApp) executeBatch(txs [][]byte) error {
batch := app.state.db.NewBatch()
defer batch.Close()
for _, tx := range txs {
key, value, err := parseTx(tx)
if err != nil {
continue
}
dataKey := append(dataPrefix, []byte(key)...)
batch.Set(dataKey, []byte(value))
}
return batch.WriteSync()
}
```

2. 캐싱 구현

```
import "github.com/hashicorp/golang-lru"
```

```go
type KVStoreApp struct {
// ... 기존 필드들
cache *lru.Cache
}

func NewKVStoreApp(dbDir string) *KVStoreApp {
cache, _ := lru.New(1000) // 1000개 항목 캐시
return &KVStoreApp{
// ... 기존 초기화
cache: cache,
}
}

func (app *KVStoreApp) getCachedValue(key string) ([]byte, bool) {
if val, ok := app.cache.Get(key); ok {
return val.([]byte), true
}
return nil, false
}

func (app *KVStoreApp) setCachedValue(key string, value []byte) {
app.cache.Add(key, value)
```

```
}
```

10. 다음 단계

이 실습을 완료했다면 다음 주제들을 학습해보세요:

1. **P2P 네트워킹**: 네트워크 통신 메커니즘
2. **상태 관리**: 고급 상태 관리 기법
3. **모니터링**: 애플리케이션 모니터링 방법

요약

이 실습을 통해 다음을 학습했습니다:

- ✅ **ABCI 애플리케이션의 기본 구조**
- ✅ **핵심 ABCI 메서드 구현**
- ✅ **상태 관리 및 영속성**
- ✅ **Query 메서드를 통한 다양한 조회 기능**
- ✅ **트랜잭션 검증 및 실행**
- ✅ **이벤트 시스템 활용**
- ✅ **클라이언트 연동 방법**
- ✅ **테스트 작성 및 성능 최적화**

이제 여러분만의 블록체인 애플리케이션을 구축할 준비가 되었습니다!